□ 1-HOUR CRASH PLAN (Follow this strictly)

0–15 min → Core Concepts (must understand)

15–45 min → Practice ALL 4 Tasks

● **45–60 min → Revise + Memorize Patterns**

PART 1: CORE CONCEPTS (SUPER SIMPLE)

1. Class & Object

- **Class = blueprint**
- **Object = real instance**

```
class Student:
    pass

s1 = Student()
```

---

2. Constructor (__init__)

- Runs automatically when object is created
- Used to assign values

```
def __init__(self, name):
    self.name = name
```

☞ self = current object

---

3. Attributes & Methods

- **Attributes = variables**
- **Methods = functions inside class**

---

4. Inheritance

- Child class uses parent class

class Child(Parent):

- Use super() to call parent constructor

---

5. Encapsulation (VERY IMPORTANT ⚠□)

- Private variables → __name
- Cannot access directly outside class

---

6. Method Overriding (Exam favorite)

- Child replaces parent method

🔥 PART 2: PRACTICE TASKS (EXAM READY CODE)

✅ Task 1: Basic Class

```
class Student:
    def __init__(self, name, roll_number):
        self.name = name
        self.roll_number = roll_number

    def display(self):
        print("Name:", self.name)
        print("Roll Number:", self.roll_number)

# Object
s1 = Student("Ali", 101)
s1.display()
```

💡 KEY: constructor + display

✅ Task 2: Inheritance + Average

```
class Student:
    def __init__(self, name, roll_number):
        self.name = name
        self.roll_number = roll_number

    def display(self):
```

```
        print("Name:", self.name)
        print("Roll Number:", self.roll_number)


class Exam(Student):
    def __init__(self, name, roll_number, marks):
        super().__init__(name, roll_number)
        self.marks = marks

    def average(self):
        return sum(self.marks) / len(self.marks)

    def display_result(self):
        self.display()
        print("Average Marks:", self.average())


# Objects
s1 = Exam("Ali", 101, [80, 90, 85])
s2 = Exam("Sara", 102, [75, 85, 95])

s1.display_result()
s2.display_result()
```

💡 KEY:

- super()
- list handling
- method calling

✅ Task 3: Encapsulation (IMPORTANT)

```
class BankAccount:
    def __init__(self, acc_no, balance):
        self.__account_number = acc_no
        self.__balance = balance

    def deposit(self, amount):
        self.__balance += amount

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient balance")

    def display(self):
        print("Account Number:", self.__account_number)
        print("Balance:", self.__balance)


# Object
b1 = BankAccount(12345, 5000)
b1.deposit(2000)
```

```
b1.withdraw(3000)
b1.display()
```

💡 KEY:

- __private
- access only via methods

✅ Task 4: Method Overriding

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def get_details(self):
        print("Name:", self.name)
        print("Salary:", self.salary)


class Manager(Employee):
    def __init__(self, name, salary, bonus):
        super().__init__(name, salary)
        self.bonus = bonus

    def get_details(self):  # override
        print("Manager Details:")
        print("Name:", self.name)
        print("Salary:", self.salary)
        print("Bonus:", self.bonus)
        print("Total:", self.salary + self.bonus)


class Developer(Employee):
    def __init__(self, name, salary, project):
        super().__init__(name, salary)
        self.project = project

    def get_details(self):  # override
        print("Developer Details:")
        print("Name:", self.name)
        print("Salary:", self.salary)
        print("Project:", self.project)


m = Manager("Ahmed", 80000, 15000)
d = Developer("Sara", 60000, "AI Chatbot")

m.get_details()
d.get_details()
```

💡 KEY:

- same function name
- different behavior

🚀 PART 3: QUICK REVISION (LAST 15 MIN)

Memorize THIS:

✅ `__init__` → constructor
✅ `self` → object reference
✅ `super()` → parent call
✅ `__var` → private
✅ overriding → same function name

⚡ COMMON EXAM MISTAKES (DON'T DO THESE)

❌ Forgetting `self`
❌ Not using `super()`
❌ Writing print instead of return (for average)
❌ Accessing private variables directly
❌ Wrong indentation

🎯 FINAL TIP

If you forget code in exam → just remember pattern:

1. Class
2. Constructor
3. Methods
4. Object
5. Call methods